Day 3 — REST & SOAP API Fundamentals

> **Activity packet** for participant triads. Today's job: design the **API contract** for the feature — resource modeling, methods, idempotency, versioning, error semantics — and draft TMD Section 3.

Where we are in the week

The data model exists (Day 1). The cloud topology exists (Day 2). Today the **API surface** that exposes the feature gets designed. The contract is what other teams (mobile, partner, internal callers) will integrate against. Get it right and integration is cheap; get it wrong and it ripples for years.

Inputs

- TMD Section 1 (entities; the API exposes these)
- TMD Section 2 (cloud topology; informs the network/auth boundary)
- TCD Section 3 (security/compliance; informs auth + rate limit + audit)
- TCD Section 4 (SLOs; APIs that meet latency targets shape design)
- PRD Section 6 AC (the user-visible behaviors the API enables)

REST and SOAP — the two API styles you must know

REST (the dominant style)

REST treats the system as a set of **resources** (nouns) accessed by standard HTTP methods (verbs).

Method	Purpose	Idempotent?
GET	Retrieve	Yes
POST	Create	No
PUT	Replace	Yes
PATCH	Update partially	No (typically)
DELETE	Remove	Yes

URLs reflect resources: `/dispatchers/{id}/reconciles/{id}`. Status codes carry meaning: 200, 201, 204, 400, 401, 403, 404, 409, 422, 429, 500, 502, 503.

SOAP (the older XML-based protocol)

SOAP is XML over HTTP (or other transports), with a defined envelope, headers, and contract (WSDL).

When SOAP is still the right answer:

- The integration partner only speaks SOAP (banks, government, EDI-adjacent systems)
- WS-Security or WS-ReliableMessaging is contractually required
- Partner has invested heavily in SOAP tooling; rewriting them is impractical

For **most** new features in B2B SaaS today, REST (often with JSON) is the right default. SOAP appears at integration boundaries with legacy partners. Today's TMD Section 3 should default to REST and explicitly justify any SOAP touchpoint.

The five things a TPM should be able to read in an API contract

Aspect	Question to answer
Resources	What nouns does the API expose? Are they consistent?
Methods	What verbs apply? Are they used consistently?
Idempotency	Can a client safely retry? How? (Idempotency key? Natural idempotence?)
Versioning	How does the API evolve without breaking existing callers?
Errors	What does a failure look like? Do error codes carry actionable info?

A TPM doesn't write the OpenAPI spec line by line, but should be able to **review** one and ask sharp questions.

Activity 1 — Resource Modeling Calibration

Format: Triad • **35 min** • Block 1

Purpose

Convert entities into resource designs before applying it to the triad's PRD feature.

Setup

Each triad needs the REST method-map card and blank paper for resource sketches. AI optional; provenance-log if used.

The example: a "todo list" API

The triad designs the resource model for a generic todo-list API. Specifically:

- The user has todo-lists; each list has todo-items
- A todo-item belongs to one list; can be marked done; has a title + description + due-date
- Users can share a list with collaborators

The protocol

1. **Sketch the resources** (10 min):
    - What URLs / paths represent each resource?
    - What's the relationship between users, lists, items, collaborators?
2. **Decide methods** (10 min). For each resource, what HTTP methods make sense?
3. **Identify the trickiest design choice** (10 min). Two examples:
    - Reordering items within a list — what does the request look like?
    - Sharing a list — is "share" a verb or a resource (e.g., POST /lists/{id}/shares)?
4. **Argue** (5 min). Different triads will land differently. The discussion is the calibration.

Readout (60 sec per triad)

> "Our resource for sharing was [X]. We considered [Y] but rejected because [reason]."

Deliverable

Resource sketch for the todo-list API: URLs, methods, and a decision on the "share" representation (verb vs resource).

Activity 2 — Design Your Feature's API

Format: Triad • **40 min** • Block 2

Purpose

Apply the resource modeling to your triad's PRD feature.

Setup

Each triad needs TMD Section 1 (entities), the TCD Section 4 SLOs, and an OpenAPI-style template. AI optional.

Triad protocol

1. **List the resources from your data model** (10 min). Each entity is a candidate resource. Some are sub-resources.
2. **Design the URL paths** (10 min). Use `/<plural>/{id}` for individual; nested for sub-resources.
3. **Methods + status codes** (10 min). For each path, which methods? What status codes for happy path / sad path?
4. **Document one endpoint in detail** (10 min). Pick the central endpoint; write a short OpenAPI-style entry.

Worked example — FieldPulse reconcile API

```yaml
# POST /v1/dispatchers/{dispatcher_id}/reconcile-events
summary: Submit a reconcile event for a dispatcher
parameters:
  - name: dispatcher_id
    in: path
    required: true
    schema:
      type: string
      format: uuid
request:
  headers:
    Authorization: Bearer <SSO JWT with reconcile.write scope>
    Idempotency-Key: <client-generated UUID>
  body:
    type: object
    required: [ticket_ids, submitted_at]
    properties:
      ticket_ids:
        type: array
        items: { type: string, format: uuid }
        minItems: 1
        maxItems: 100
      submitted_at:
        type: string
        format: date-time
      client_metadata:
        type: object
        properties:
          user_agent: { type: string }
          location: { type: string }
responses:
  201: Created — { id, ticket_ids, submitted_at }
  400: Validation error — { error: code, fields: [{name, reason}] }
  401: Missing/invalid token
  403: Token lacks reconcile.write scope
  409: Idempotency-Key already used with different body
  422: One or more tickets not eligible for reconcile
  429: Rate limit exceeded — Retry-After: <seconds>
```

What "good" looks like

- URLs are nouns (resources), not verbs
- Methods are used **consistently**
- Status codes are **specific** — 422 vs 400 vs 409 carry distinct meaning
- Required headers (Authorization, Idempotency-Key) are explicit

Deliverable

Resource list with URL paths, methods, and status codes, plus one detailed OpenAPI-style endpoint definition for the central feature.

Activity 3 — Idempotency, Versioning, Error Semantics

Format: Triad • **40 min** • Block 3

Purpose

Cover the three aspects most often hand-waved: idempotency, versioning, and error semantics.

Setup

Each triad needs the resource design from Activity 2, the idempotency-mechanism card, the versioning-strategy card, and the standard error-codes card.

Idempotency — the "safe to retry" guarantee

A request is idempotent if the same request, sent twice, produces the same result. Networks fail; clients retry; idempotency saves you from creating duplicate records.

Three ways to achieve idempotency:

1. **Natural idempotence** — PUT and DELETE are idempotent by default
2. **Idempotency-Key header** — client sends a unique key; server stores key + response and returns the same response on retry
3. **Conditional requests** — If-Match / If-None-Match headers gate the operation

For your feature, decide:

- Which mutating endpoints support idempotency
- How (which mechanism)
- How long the idempotency window is (typical: 24 hours)

Versioning — the "evolve without breaking" guarantee

Three common strategies:

Strategy	Where the version lives	Pros	Cons
URL path	`/v1/...` `/v2/...`	Easy to read; clear breaking changes	Code duplication
Header	`X-API-Version: 2`	URL stays clean	Less discoverable
Content negotiation	`Accept: application/vnd.acme+json;v=2`	Standard	Often poorly understood

For most B2B APIs, **URL path versioning** is the right default. Pick a strategy and document it.

Error semantics — the "what do I do now?" guarantee

The 7 most-needed error responses for any API:

Code	Meaning	Body should include
400	Bad request (validation)	Field-by-field errors with codes
401	Unauthenticated	Hint: SSO challenge or token-refresh path
403	Authenticated but unauthorized	Required scope name
404	Resource doesn't exist	Resource type + identifier
409	Conflict (idempotency / state)	What conflicted; how to resolve
422	Validated but business-rule violation	Which rule; which fields involved
429	Rate-limited	Retry-After header; current limit

Standardize the **error body shape** across the API:

```
{
  "error": {
    "code": "ticket_not_eligible",
    "message": "Ticket cannot be reconciled in current state",
    "fields": [{"name": "ticket_ids[3]", "reason": "ticket already
        reconciled"}]
  },
  "request_id": "req_01H...",
  "documentation_url": "https://docs.../errors/ticket_not_eligible"
}
```

Triad protocol

1. **Idempotency** (15 min). For each mutating endpoint, decide approach + window.

2. **Versioning** (10 min). Pick strategy; document in Section 3.
3. **Error semantics** (15 min). Define error body shape; list the 7 most-needed codes for your endpoints.

Deliverable

A documented idempotency approach per mutating endpoint, a chosen versioning strategy, and a standardized error body shape covering the 7 codes.

Activity 4 — REST vs SOAP + AI Critique

Format: Triad • **45 min** + Wrap • Block 4

Purpose

Decide whether any part of the API surface needs SOAP. Use AI to critique the contract.

Setup

Each triad needs TCD Section 2 (integration map), the API contract from Activities 2–3, and the AI critique prompt. AI required.

When to consider SOAP

For each external integration in your TCD Section 2, ask:

- Is the partner SOAP-only?
- Is WS-Security required (rare; banks, government)?
- Is there a WSDL the partner expects us to consume / produce?

For most FieldPulse-shaped features, the answer is **no**. SOAP appears occasionally for legacy enterprise integrations. Document the call honestly.

Triad protocol — SOAP question (10 min)

Walk the integration map. For each, default to REST. If SOAP is required for one, document it as an exception in Section 3.

AI critique prompt

```
Role: API design reviewer with strong REST opinions.
Context: <paste the URL paths, methods, status codes, error body
         shape, idempotency approach, versioning strategy>
Task: Identify the top 5 issues in this API contract.
Constraints:
  - Be specific: cite the path, method, or convention being violated
  - For each issue, suggest the convention or correction
  - Avoid pedantic preferences (singular/plural is fine either way
    if consistent)
Format: Numbered — Issue / What's wrong / Fix.
```

Triad protocol — AI critique (20 min)

1. **Run the prompt** (10 min). Capture the 5 issues.
2. **Adopt / defer / reject** (10 min). Same Week-3 discipline.

Final polish (10 min)

Update Section 3. Add provenance note.

Deliverable

Polished TMD Section 3 incorporating adopted AI findings, any SOAP exceptions documented, and a provenance note for AI prompts.

Wrap (last 15 min)

Each triad shares:

- One endpoint they're proudest of (with a one-line "why")
- One idempotency choice they made
- One error code they had to think hardest about

End-of-day checkpoint

Each triad ends Day 3 with:

- [x] Resource design with URL paths
- [x] Methods + status codes per endpoint
- [x] One detailed endpoint (OpenAPI-style)
- [x] Idempotency approach for mutating endpoints
- [x] Versioning strategy
- [x] Error body shape + the 7 most-needed codes
- [x] SOAP exceptions documented (if any)

- [x] AI provenance log entry
- [x] TMD Section 3 drafted